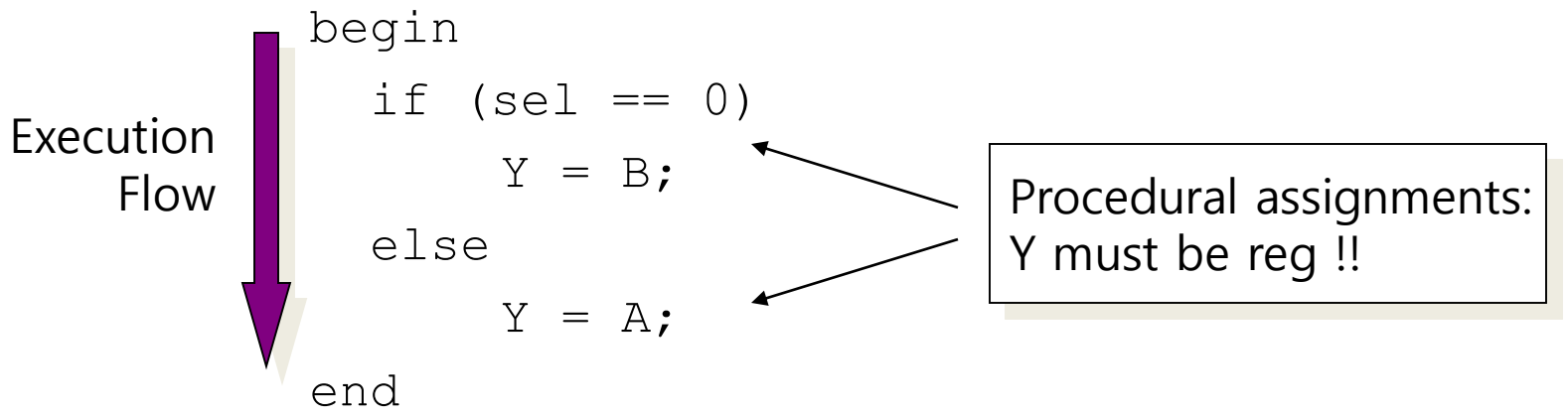
Behavioral Model - Procedures (i)

- Procedures = sections of code that we know they execute sequentially
- Procedural statements = statements inside a procedure (they execute sequentially)
- e.g. another 2-to-1 mux implem:



Behavioral Model - Procedures (ii)

- Modules can contain any number of procedures
- Procedures execute in parallel (in respect to each other) and ..
- .. can be expressed in two types of blocks:
  - initial → they execute only once
  - always → they execute for ever (until simulation finishes)

“Initial” Blocks

- Start execution at sim time zero and finish when their last statement executes

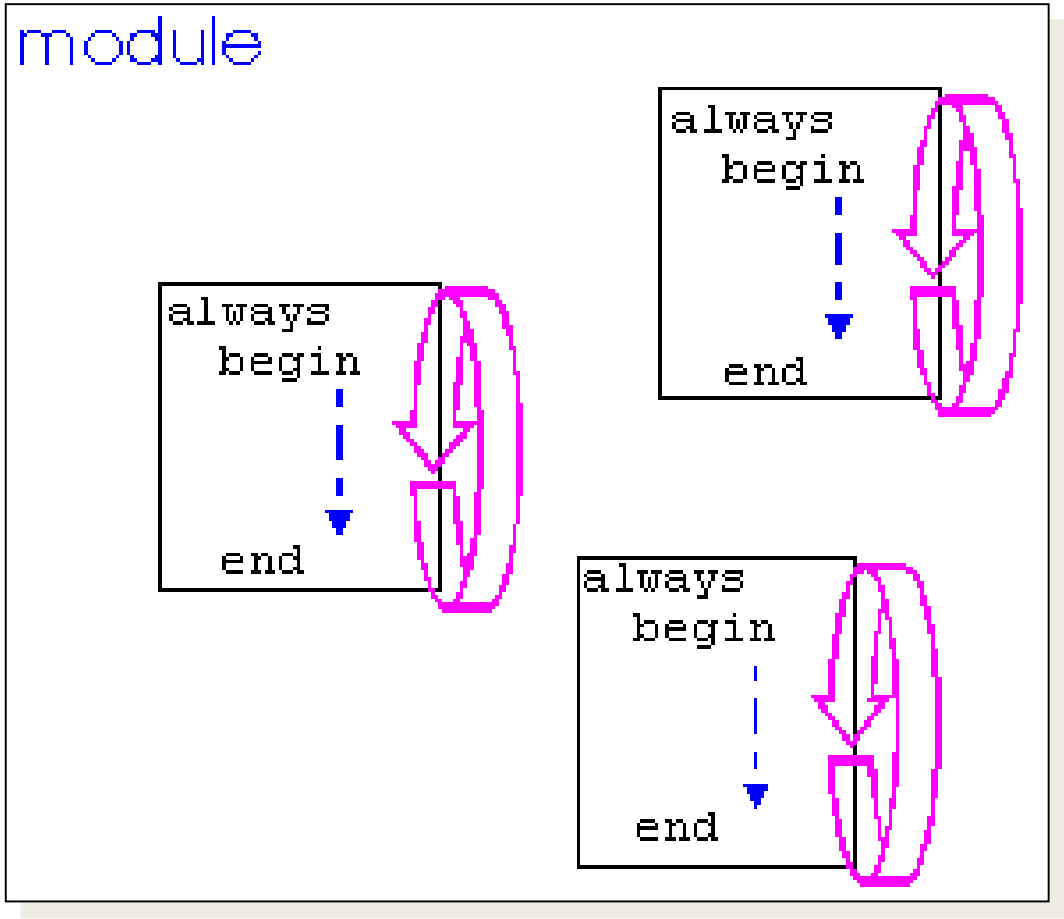
```
module nothing;

initial
  $display(“I’m Groot”);

initial begin
  #50;
  $display(“I’m Steve Rogers”);
end

endmodule
```


Will be displayed at sim time 0

Will be displayed at sim time 50

"Always" Blocks

- Start execution at sim time zero and continue until sim finishes



Procedural Assignment with always

- Procedural assignment allows an alternative, often higher-level, behavioral description of combinational logic
- Two structured procedure statements: `initial` and `always`
- Supports richer, C-like control structures such as `if`, `for`, `while`, `case`

```
module mux_2_to_1(a, b, out,
                  outbar, sel);
  input a, b, sel;
  output out, outbar;

  reg out, outbar;

  always @ (a or b or sel)

  begin
    if (sel) out = a;
    else out = b;

    outbar = ~out;

  end

endmodule
```

Exactly the same as before.

Anything assigned in an `always` block must *also* be declared as type `reg` (next slide)

Conceptually, the `always` block runs *once* whenever a signal in the sensitivity list changes value

Statements within the `always` block are executed sequentially. Order matters!

Surround multiple statements in a single `always` block with `begin/end`.

Verilog Registers

- In digital design, registers represent memory elements
- Digital registers need a clock to operate and update their state on certain phase or edge
- Registers in Verilog should not be confused with hardware registers
- In Verilog, the term register (`reg`) simply means a variable that can hold a value
- Verilog registers don't need a clock and don't need to be driven like a net. Values of registers can be changed anytime in a simulation by assuming a new value to the register

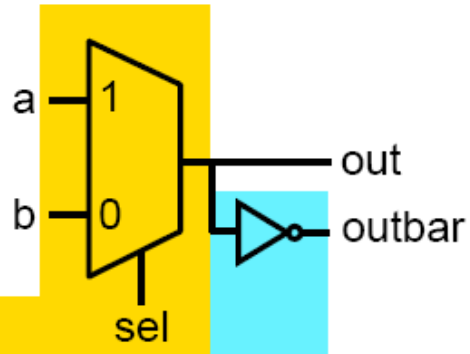
Mix-and-Match Assignments

- Procedural and continuous assignments can (and often do) co-exist within a module
- Procedural assignments update the value of `reg`. The value will remain unchanged till another procedural assignment updates the variable. This is the main difference with continuous assignments in which the right hand expression is constantly placed on the left-side

```
module mux_2_to_1(a, b, out,
                  outbar, sel);
  input a, b, sel;
  output out, outbar;
  reg out;

  always @ (a or b or sel)
  begin
    if (sel) out = a;
    else out = b;
  end

  assign outbar = ~out;

endmodule
```

procedural description

continuous description



if, case Statement

- `case` and `if` may be used interchangeably to implement conditional execution within `always` blocks
- `case` is easier to read than a long string of `if...else` statements

```
module mux_2_to_1(a, b, out,
                  outbar, sel);
  input a, b, sel;
  output out, outbar;
  reg out;

  always @ (a or b or sel)
  begin
    if (sel) out = a;
    else out = b;
  end

  assign outbar = ~out;

endmodule
```

```
module mux_2_to_1(a, b, out,
                  outbar, sel);
  input a, b, sel;
  output out, outbar;
  reg out;

  always @ (a or b or sel)
  begin
    case (sel)
      1'b1: out = a;
      1'b0: out = b;
    endcase
  end

  assign outbar = ~out;

endmodule
```

Note: Number specification notation: <size>'<base><number>
(4'b1010 if a 4-bit binary value, 16'h6cda is a 16 bit hex number, and 8'd40 is an 8-bit decimal value)

Procedural Statements: if

```
if (expr1)
        true_stmt1;

else if (expr2)
        true_stmt2;
..
else
        def_stmt;
```

E.g. 4-to-1 mux:

```
module mux4_1(out, in, sel);
output out;
input [3:0] in;
input [1:0] sel;

reg out;
wire [3:0] in;
wire [1:0] sel;

always @(in or sel)
        if (sel == 0)
                out = in[0];
        else if (sel == 1)
                out = in[1];
        else if (sel == 2)
                out = in[2];
        else
                out = in[3];
endmodule
```

Procedural Statements: case

```
case (expr)

item_1, .., item_n:      stmt1;
item_n+1, .., item_m:   stmt2;
..
default:                 def_stmt;

endcase
```

E.g. 4-to-1 mux:

```
module mux4_1(out, in, sel);
output out;
input [3:0] in;
input [1:0] sel;

reg out;
wire [3:0] in;
wire [1:0] sel;

always @(in or sel)
        case (sel)
        0: out = in[0];
        1: out = in[1];
        2: out = in[2];
        3: out = in[3];
        endcase
endmodule
```

Procedural Statements: for

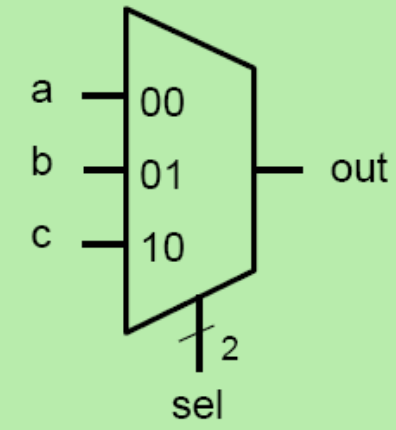
for (init_assignment; cond; step_assignment)
    stmt;

E.g.

```
module count(Y, start);
output [3:0] Y;
input start;

reg [3:0] Y;
wire start;
integer i;

initial
        Y = 0;          // Vivado ok. DC_NXT not ok

always @(posedge start)
        for (i = 0; i < 3; i = i + 1)
                #10 Y = Y + 1;
endmodule
```

Dangers of Verilog: Incomplete Specification

Goal:



3-to-1 MUX
('11' input is a don't-care)

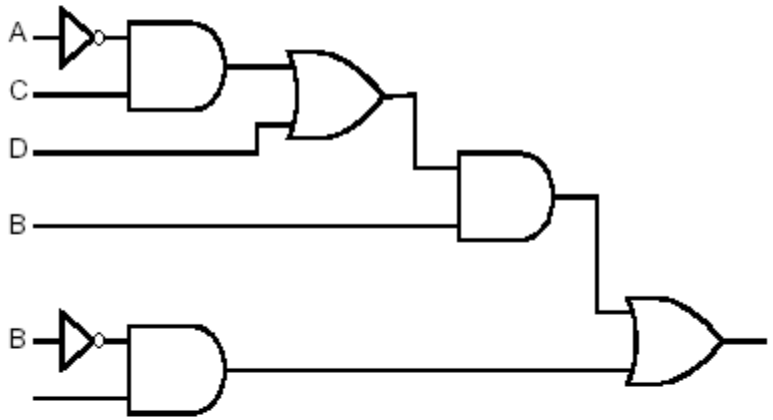
Proposed Verilog Code:

```
module maybe_mux_3to1(a, b, c,
                      sel, out);
  input [1:0] sel;
  input a,b,c;
  output out;
  reg out;

  always @(a or b or c or sel)
  begin
    case (sel)
      2'b00: out = a;
      2'b01: out = b;
      2'b10: out = c;
    endcase
  end
endmodule
```

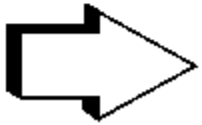
Is this a 3-to-1 multiplexer?

Language Directed View

- Inferring combinational logic
  - every combination of inputs produces an output
  - every execution path performs an assignment to every signal



- Inferring sequential logic
  - a signal is not assigned in some execution path

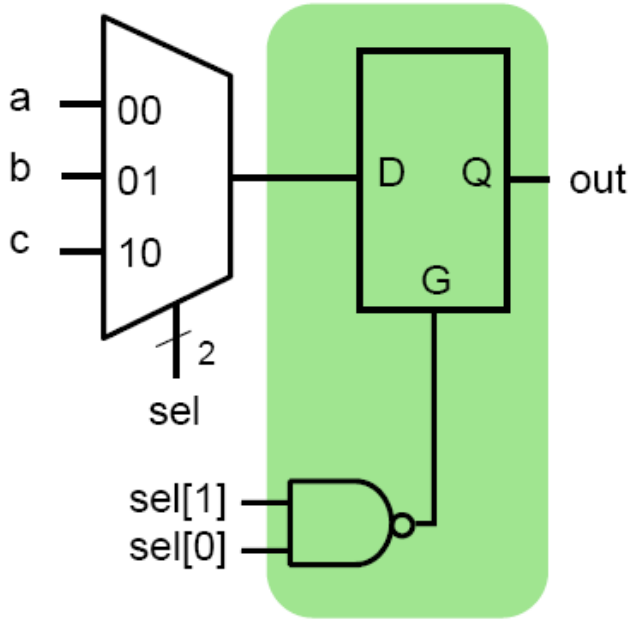


remember the value, i.e., storage

Incomplete Specification Infers Latches

```
module maybe_mux_3to1(a, b, c,
                      sel, out);
  input [1:0] sel;
  input a,b,c;
  output out;
  reg out;

  always @(a or b or c or sel)
  begin
    case (sel)
      2'b00: out = a;
      2'b01: out = b;
      2'b10: out = c;
    endcase
  end
endmodule
```

Synthesized Result:



if `out` is not assigned during any pass through the always block, then the previous value must be retained!

- Latch memory "latches" old data when G=0 (we will discuss latches later)
- In practice, we almost *never* intend this

Avoiding Incomplete Specification

- Precede all conditionals with a default assignment for all signals assigned within them…

```
always @(a or b or c or sel)
  begin
    out = 1'bx;
    case (sel)
      2'b00: out = a;
      2'b01: out = b;
      2'b10: out = c;
    endcase
  end
endmodule
```

```
always @(a or b or c or sel)
  begin
  case (sel)
      2'b00: out = a;
      2'b01: out = b;
      2'b10: out = c;
      default: out = 1'bx;
    endcase
  end
endmodule
```

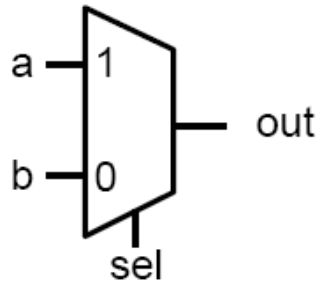
- …or, fully specify all branches of conditionals and assign all signals from all branches
  - For each `if`, include `else`
  - For each `case`, include `default`

The Sequential always Block

- Edge-triggered circuits are described using a sequential `always` block

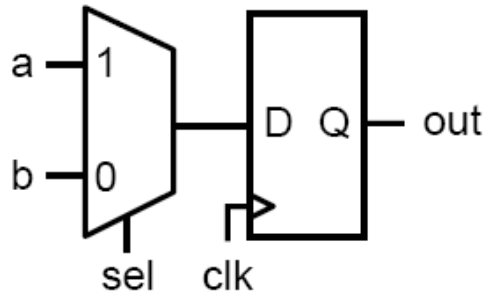
Combinational

```
module combinational(a, b, sel,
                     out);
  input a, b;
  input sel;
  output out;
  reg out;

  always @ (a or b or sel)
  begin
    if (sel) out = a;
    else out = b;
  end

endmodule
```



Sequential

```
module sequential(a, b, sel,
                  clk, out);
  input a, b;
  input sel, clk;
  output out;
  reg out;

  always @ (posedge clk)
  begin
    if (sel) out <= a;
    else out <= b;
  end

endmodule
```



Blocking vs. Nonblocking Assignments

- Verilog supports two types of assignments within `always` blocks, with subtly different behaviors.
- *Blocking assignment:* evaluation and assignment are immediate

```
always @ (a or b or c)
begin
  x = a | b;          1. Evaluate a | b, assign result to x
  y = a ^ b ^ c;      2. Evaluate a^b^c, assign result to y
  z = b & ~c;         3. Evaluate b&(~c), assign result to z
end
```

- *Nonblocking assignment:* all assignments deferred until all right-hand sides have been evaluated (end of simulation timestep)

```
always @ (a or b or c)
begin
  x <= a | b;         1. Evaluate a | b but defer assignment of x
  y <= a ^ b ^ c;     2. Evaluate a^b^c  but defer assignment of y
  z <= b & ~c;        3. Evaluate b&(~c) but defer assignment of z
end                   4. Assign x, y, and z with their new values
```

- Sometimes, as above, both produce the same result. Sometimes, not!

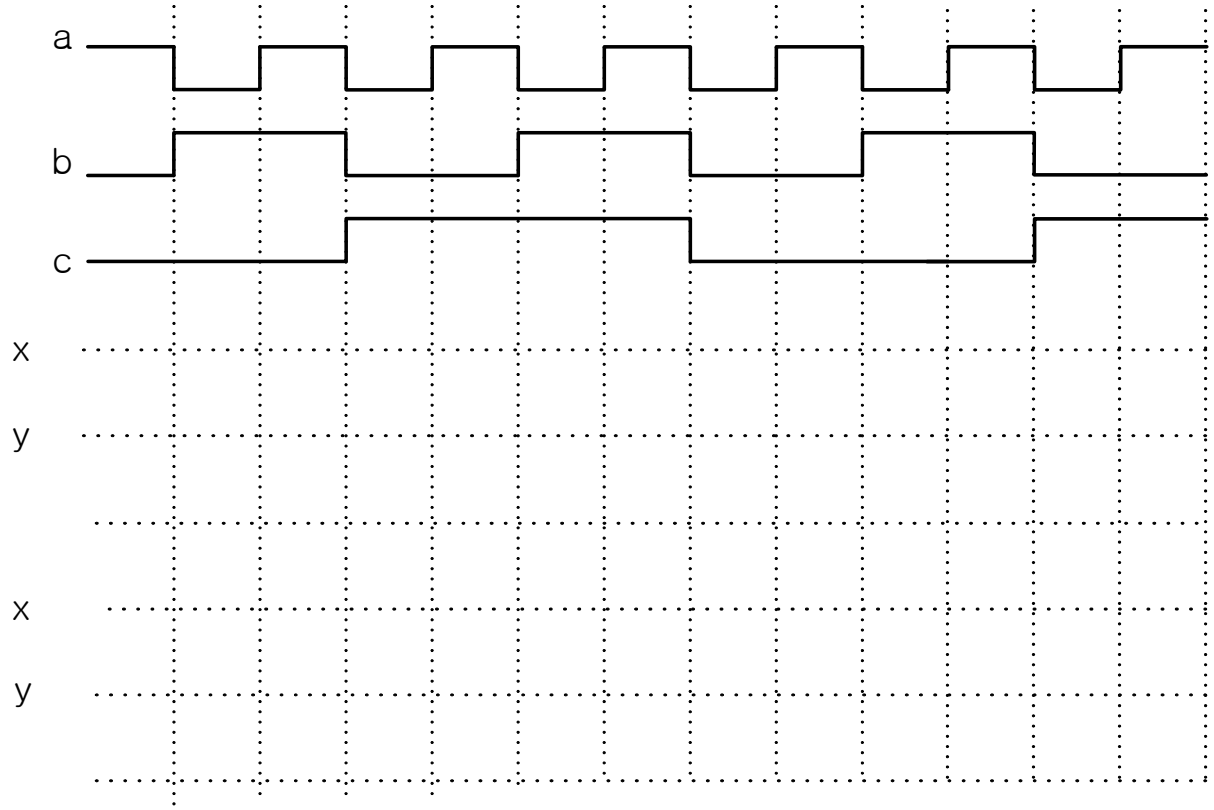
Blocking and Nonblocking: how to predict behavior?

1. A procedure takes zero time to schedule events.
2. All statements are executed sequentially.
3. Blocking assignments take effect immediately.
4. Nonblocking assignments take effect after the delta delay.
5. A procedure executes only when an event occurs on the signals in the sensitivity list.

Blocking vs. Nonblocking

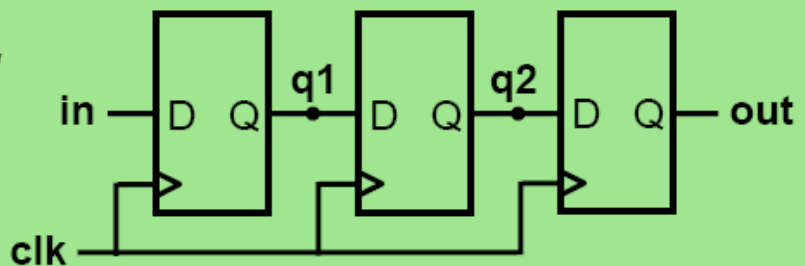
```
always @ (a, b, c)
begin
  x = a & b;
  y = x | c;
end
```

```
always @ (a, b, c)
begin
  x <= a & b;
  y <= x | c;
end
```



Assignment Styles for Sequential Logic

Flip-Flop Based Digital Delay Line

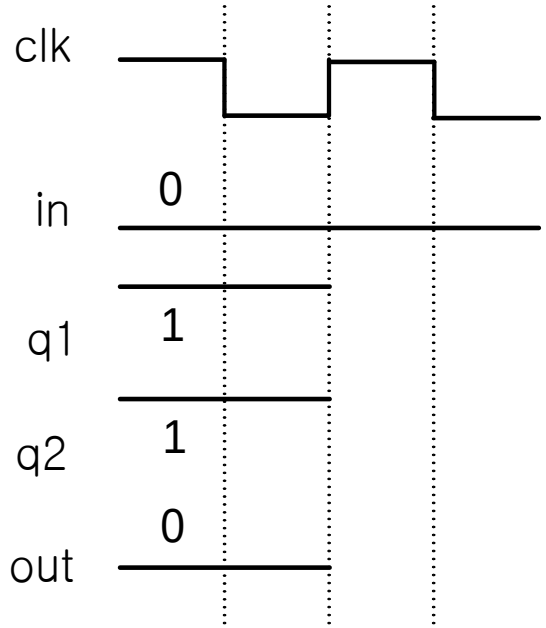


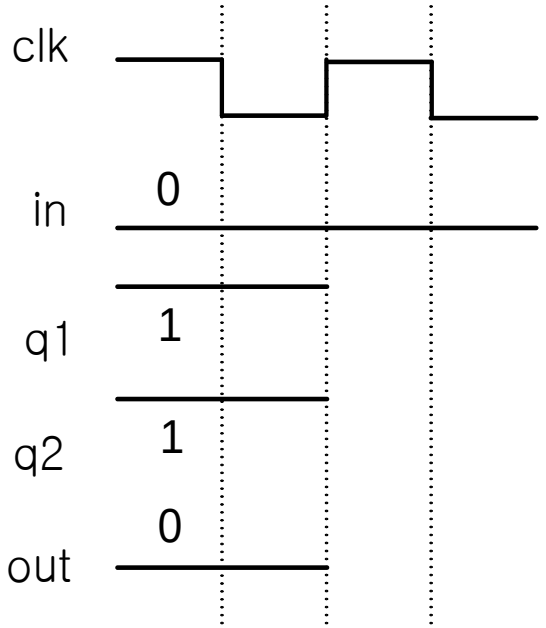
- **Will nonblocking and blocking assignments both produce the desired result?**

```
module nonblocking(in, clk, out);
  input in, clk;
  output out;
  reg q1, q2, out;

  always @ (posedge clk)
  begin
    q1 <= in;
    q2 <= q1;
    out <= q2;
  end

endmodule
```

```
module blocking(in, clk, out);
  input in, clk;
  output out;
  reg q1, q2, out;

  always @ (posedge clk)
  begin
    q1 = in;
    q2 = q1;
    out = q2;
  end

endmodule
```

Assignment Styles for Sequential Logic

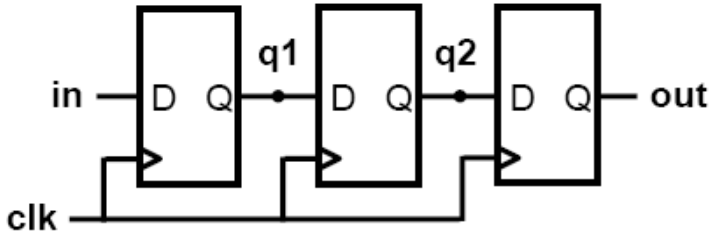




Use Nonblocking for Sequential Logic

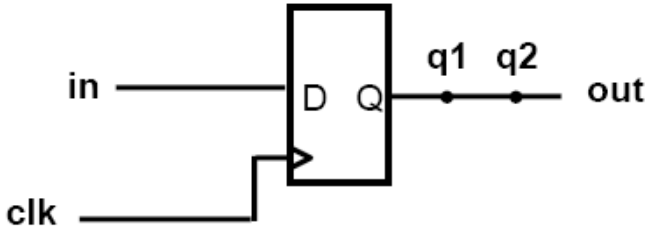
```
always @ (posedge clk)
begin
  q1 <= in;
  q2 <= q1;
  out <= q2;
end
```

"At each rising clock edge, *q1*, *q2*, and *out* simultaneously receive the old values of *in*, *q1*, and *q2*."




```
always @ (posedge clk)
begin
  q1 = in;
  q2 = q1;
  out = q2;
end
```

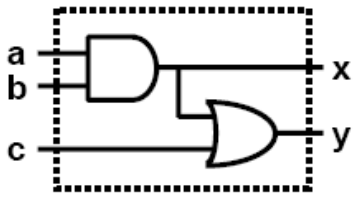
"At each rising clock edge, *q1* = *in*. After that, *q2* = *q1* = *in*. After that, *out* = *q2* = *q1* = *in*. Therefore *out* = *in*."



- Blocking assignments do not reflect the intrinsic behavior of multi-stage sequential logic
- Guideline: use nonblocking assignments for sequential `always` blocks

Use Blocking for Combinational Logic

Blocking Behavior	a b c x y
(Given) Initial Condition	1 1 0 1 1
a changes; `always` block triggered	0 1 0 1 1
`x = a & b;`	0 1 0 0 1
`y = x | c;`	0 1 0 0 0




```
module blocking(a,b,c,x,y);
  input a,b,c;
  output x,y;
  reg x,y;

  always @ (a or b or c)
  begin
    x = a & b;
    y = x | c;
  end

endmodule
```

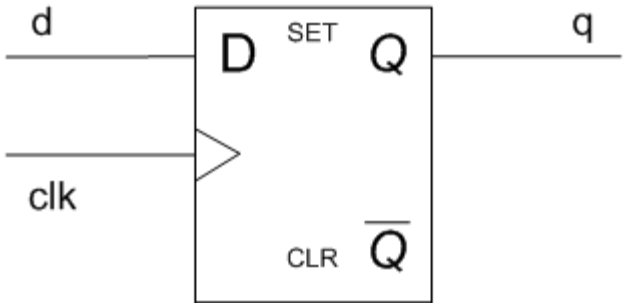
Nonblocking Behavior	a b c x y	*Deferred*
(Given) Initial Condition	1 1 0 1 1	
a changes; `always` block triggered	0 1 0 1 1	
`x <= a & b;`	0 1 0 1 1	x<=0
`y <= x | c;`	0 1 0 1 1	x<=0, y<=1
Assignment completion	0 1 0 0 1	

```
module nonblocking(a,b,c,x,y);
  input a,b,c;
  output x,y;
  reg x,y;

  always @ (a or b or c)
  begin
    x <= a & b;
    y <= x | c;
  end

endmodule
```

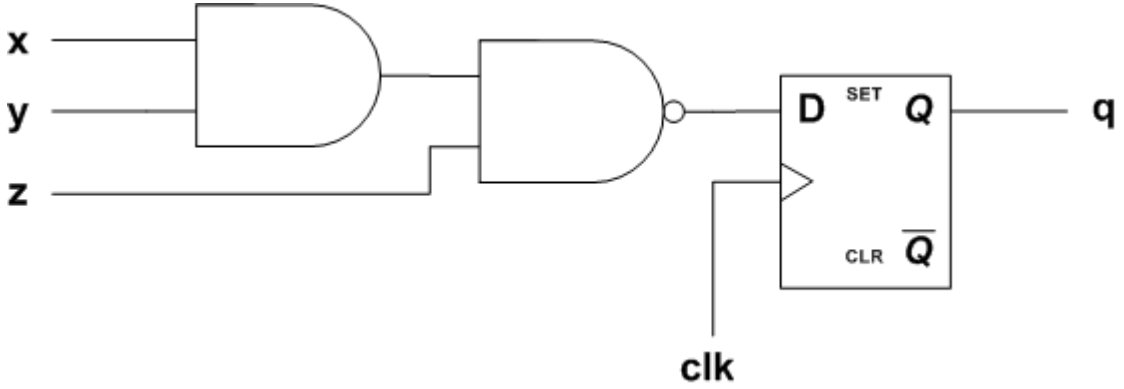
- Nonblocking and blocking assignments will synthesize correctly. Will both styles simulate correctly?
- Nonblocking assignments do not reflect the intrinsic behavior of multi-stage combinational logic
- While nonblocking assignments can be hacked to simulate correctly (expand the sensitivity list), it's not elegant
- Guideline: use blocking assignments for combinational `always` blocks

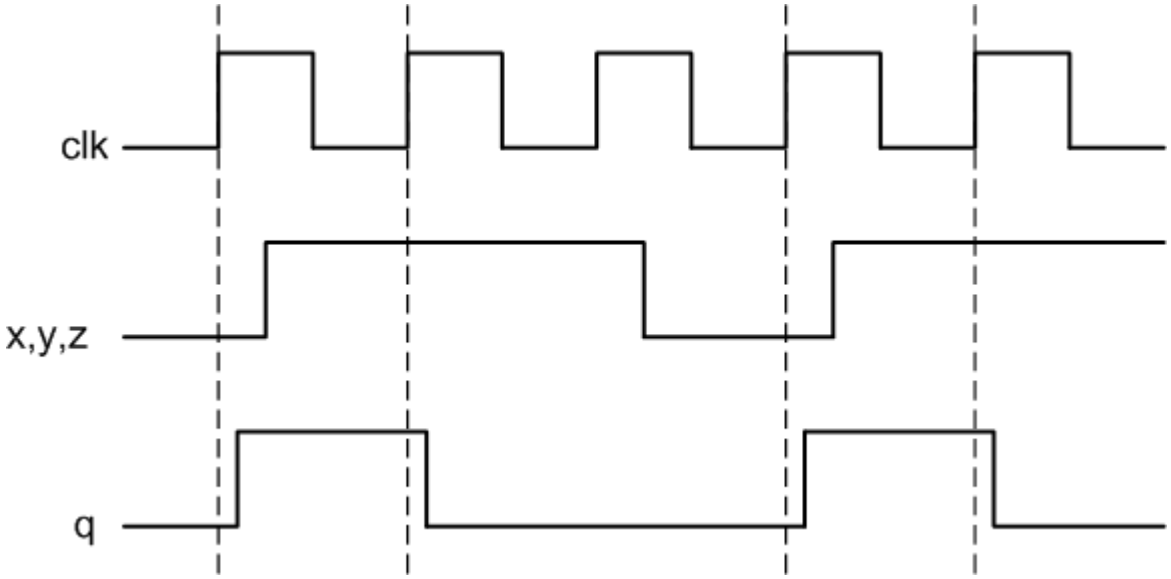
Flip-flop Verilog code

```
always @ (posedge clk)
begin
    q <= d;
end
```



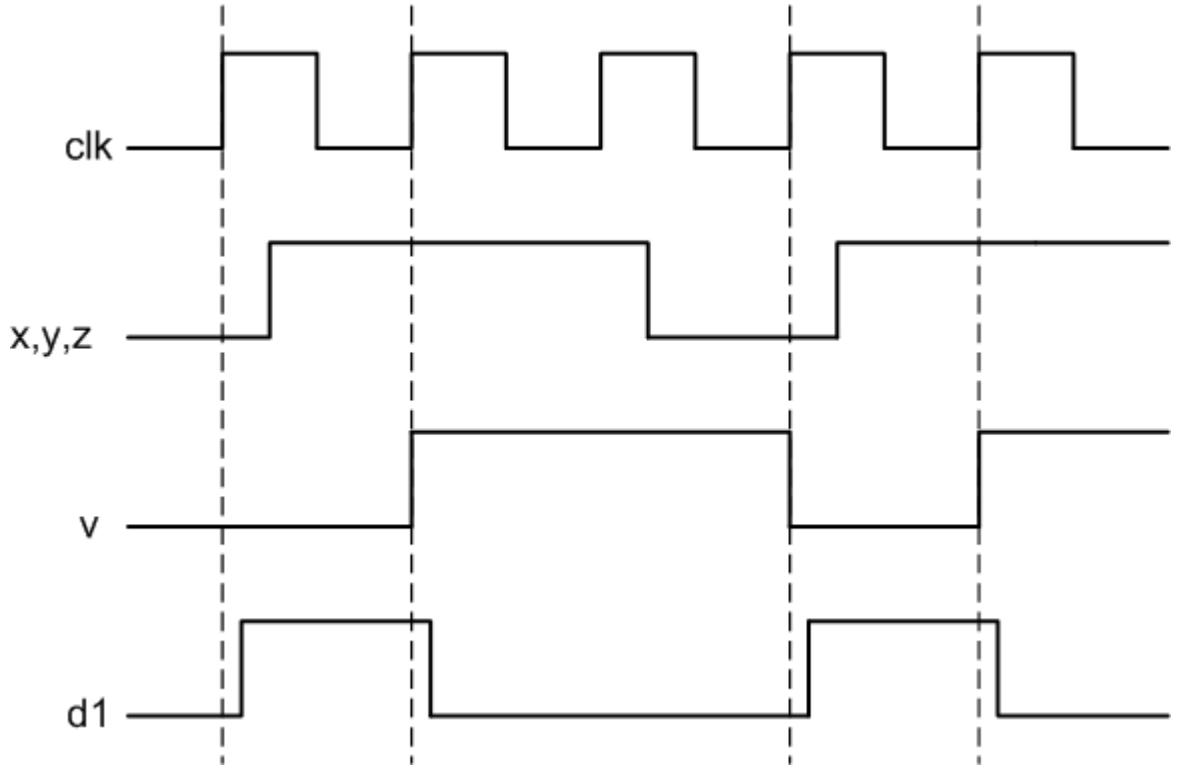
Blocking vs. Nonblocking: flip-flop





Blocking vs. Nonblocking: flip-flop

```
always @ (posedge clk)
begin
    v = x & y;
    d1 <= v ~& z;
end;
```



Blocking vs. Nonblocking: flip-flop

```
always @ (posedge clk)
begin
    s <= x & y;
    d2 <= s ~& z;
end;
```